pycognition: A Dual-Process Cognitive Architecture Integrating Deterministic Procedural Execution and Deliberative LLM Synthesis

Oleh Riabtsev
Referat Forschung, Nachwuchsförderung und Transfer (RFNT), Universität Vechta, Germany
ORCID: 0009-0003-7848-6418 | Software License: LGPL-3.0-or-later

ABSTRACT

This paper introduces **pycognition**, an open-source neurosymbolic framework formalizing dual-process cognitive dynamics for autonomous software agents. Contemporary agent frameworks uniformly route deterministic and repetitive sub-tasks through stochastic foundation model APIs, incurring substantial latency, financial overhead, and execution drift. pycognition addresses this efficiency gap via a segregated execution topology: an ultra-fast procedural tier (System 1) executing validated Python scripts, and a deliberative tier (System 2) orchestrating frontier LLM inference. The Skill Crystallization pipeline compiles verified reasoning traces into strongly typed modules, verified via Abstract Syntax Tree (AST) safety inspection and sandboxed unit testing via `pytest`. An empirical trial across 100 heterogeneous workloads confirmed a rigorous three-tier performance model. Operating akin to an adaptive JIT compiler, the architecture explicitly accounts for script development latency (mean 7,192.20 ms). The per-skill break-even point is reached at 4 invocations (ideal) and 5 invocations (accounting for the 36.2% sandbox rejection overhead), followed by an asymptotic speedup of 11,326.7x. System-wide, algorithmic response time decreased by 44.7% (155.46 s vs. 281.13 s), while cumulative token expenditure dropped immediately by 38.0%, scaling to 71.4% procedural coverage in the final quartile.

Keywords: Cognitive Architectures, Autonomous Agents, Dual-Process Theory, Program Synthesis, AST Validation, Amortized Inference, JIT Compilation, Break-Even Point, pycognition.

1. INTRODUCTION

The deployment of LLM agents across enterprise environments has exposed a fundamental inefficiency between natural language generative flexibility and deterministic algorithmic execution. Standard paradigms (ReAct [6], Plan-and-Solve [9], AutoGen [10]) utilize stochastic autoregressive inference as an all-purpose runtime. Algorithmic transformations, log parsing, and numerical calculations are repeatedly offloaded to cloud model APIs alongside strategic planning, incurring linear and quadratic token costs, round-trip latencies of up to 5,000 ms, and non-deterministic regressions.

Dual-Process Theory [1], [2] resolves this trade-off: **System 1** governs rapid procedural routines, while **System 2** allocates intensive deliberation to novel domains. The **pycognition** framework transfers this principle to software agents as an adaptive JIT compiler: encountering new task classes, the agent invests System 2 resources into synthesizing deterministic Python modules, verifies their safety, and registers them into procedural memory.

2. BACKGROUND AND RELATED WORK

The partition between declarative representations and procedural rules has historical foundations in ACT-R [3] and Soar [4]. Amortized inference [15] established the validity of replacing costly iterative search with cached evaluation operators. Foundation models (Codex [12], DeepSeek-Coder [13]) and agents like Voyager [11] demonstrated skill accumulation, but lacked deterministic AST sanitization, automated unit test generation [14], and rigorous multi-tier break-even modeling.

3. PROPOSED METHODOLOGY AND ARCHITECTURE

An incoming computational task $T = \langle \mathrm{id}, p, x, \mu \rangle$ is arbitrated via `MetaArbiter` maximizing expected utility $U(a \mid T)$ based on vector similarity $\gamma(T, \Sigma)$ with threshold $\theta = 0.55$. The crystallization operator Ψ synthesizes verified subroutines guarded by `ASTSecurityValidator` and an isolated subprocess `pytest` sandbox ($t = 15$ s).

4. IMPLEMENTATION AND EMPIRICAL EVALUATION

4.1. Three-Tier Latency and Break-Even Modeling (N = 100)

- **Tier 1. Marginal Latency and Asymptotic Acceleration:** A compiled System 1 routine executes in $\overline{L}_{S_1} = 0.292$ ms. A baseline algorithmic LLM call averages $L_{\text{base}} = 3\ 307.41$ ms. The marginal speedup is **11,326.7x** (19,103.9x across all System 2 calls).
- **Tier 2. Script Development Costs and Per-Skill Break-Even:** Synthesis and sandbox testing consume an average of $L_{\text{cryst}} = 7\ 192.20$ ms (cold-start total: 10 499.60 ms).
  The amortization condition across k invocations is:

$$k \geq 1 + \frac{L_{\text{cryst}} - \overline{L}_{S_1}}{L_{\text{base}} - \overline{L}_{S_1}} = 1 + \frac{7\ 192.20 - 0.292}{3\ 307.41 - 0.292} \approx 3.175 \Longrightarrow k = 4 \text{ calls}$$

  Accounting for the 36.2% sandbox rejection overhead ($L_{\text{cryst,eff}} = 11\ 267.78$ ms), effective break-even occurs at $k_{\text{eff}} \geq 4.407 \Longrightarrow k = 5$ calls.
  Effective speedup progression: $S(1) = 0.31\times$; $S(2) = 0.63\times$; $S(4) = 1.26\times$ (break-even reached); $S(10) = 3.15\times$; $S(50) = 15.73\times$; $S(100) = 31.41\times$.
- **Tier 3. Workload-Level Progression across 100 Tasks:**
  Baseline uncompiled LLM time: 471.66 s (281.13 s algorithmic). In pycognition, asynchronous task response time dropped to **155.46 s (1.81x speedup, -44.7% execution time)**. Synchronous wall-clock time including 47 crystallization cycles (338.03 s) totaled 493.49 s on algorithmic tasks. In Quartile 4, synchronous wall-clock speedup reached **0.96x (2.84x execution speedup)**, with full workload break-even horizon converging at $N^* \approx 207$ tasks.

Table 2. Realistic Empirical Performance Matrix (N = 100 Tasks)

Performance Metric	Pure Frontier LLM Baseline	pycognition Dual-Process	Realistic Empirical Delta
Marginal Execution Latency	3,307.41 ms (algo)	0.292 ms (System 1)	Sub-millisecond execution
Script Synthesis & Testing Latency	0 ms	7,192.20 ms	JIT compilation investment
Total Cold-Start Latency	3,307.41 ms	10,499.60 ms	Initial learning overhead (0.31x)
Per-Skill Break-Even Point	N/A	k = 4 (ideal) / k = 5 (effective)	Amortized after 4–5 calls
Marginal Speedup Factor	1.0x	11,326.7x (algo) / 19,103.9x (overall)	Asymptotic performance limit
85 Algorithmic Tasks Execution Time	281.13 s	155.46 s (asynchronous)	1.81x speedup (-44.7% time)
Synchronous Wall-Clock Time	281.13 s	493.49 s	Learning phase at N=100
System Break-Even Horizon N*	N/A	N* ≈ 207 tasks	Full cluster amortization
Token Expenditure (100 Tasks)	87,061 tokens	53,978 tokens	-38.0% immediate reduction
System 1 Share in Quartile 4 (Q4)	0.0%	71.4% (15 of 21 algorithmic)	Sustained consolidation
Crystallization Sandbox Yield	N/A	63.8% (30 passed / 17 blocked)	100% System 1 memory fidelity

5. DISCUSSION AND LIMITATIONS

Filtering 36.2% of candidate subroutines prevented silent regressions caused by degrees-of-freedom mismatches in variance (6 z-score cases), boundary flaws in IPv4 parsing (4 cases), and floating-point rounding errors. Explicitly accounting for synthesis latency (7.19 s) provides an honest engineering baseline, indicating that compilation should operate as an asynchronous background task. Lexical TF matching exhibited synonym fragmentation and cross-lingual dilution, establishing the requirement for dense multilingual embeddings.

6. CONCLUSION AND FUTURE WORK

pycognition demonstrates the empirical viability of JIT compilation for cognitive agents. Synthesis investments amortize within 4–5 task repetitions, achieving an 11,326.7x marginal speedup, a 44.7% execution time reduction, and immediate 38.0% token savings. Future work focuses on self-healing reflection loops and dense embedding deduplication.

7. REFERENCES

[1] D. Kahneman, *Thinking, Fast and Slow*. New York, NY, USA: Farrar, Straus and Giroux, 2011.

[2] J. S. B. T. Evans and K. E. Stanovich, "Dual-process theories of higher cognition: Advancing the debate," *Perspectives on Psychological Science*, 2013.

[3] J. R. Anderson, *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, 2007.

[6] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[8] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.

[11] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," in *Proc. NeurIPS*, 2023.

[15] S. J. Gershman and N. D. Goodman, "Amortized inference in probabilistic reasoning," in *Proc. CogSci*, 2014.